

Exercise: Visualizing Associations and Mapping Aesthetics

Overview

This exercises provides some practice creating data visualizations to communicate associations among variables and is designed to be completed in 75 minutes or fewer. Although you should have time to complete the parts, if you are unable, *as with all exercises, you are encouraged to complete it outside of class time* so that you are able to incorporate your experiences and knowledge into future exercises. You may need to consult course reading materials located at the course site as some elements may not have been covered in the basic content contained in associated videos.

This exercise assumes you have an understanding of `{dplyr}` functions like `select()`, `filter()`, `mutate()`, and `summarize()` and that you have been practicing using these functions as part of your learning and project progressed. Also assumed is a basic understanding of `{ggplot2}` functions like `ggplot()`, `geom_point()`, as `aes()`.

This exercise focuses on:

- Intentional File Creation and File Management
- Creating point plots
- Mapping variables to aesthetics (not setting)
- Workflow and Reproducibility

This exercise uses:

- Your RStudio Git version-control project for your team project (presumably completed)
- `{here}`, `{dplyr}`, `{ggplot2}` functions and functions from relevant Base R libraries (e.g., `readRDS()`, `saveRDS()`, `read.csv()`, etc.)
- Your knowledge from past in-class exercises, videos, homework, etc.
- Some data that you are using for your team project.

Note: If your RStudio project for your team is not created, you will work in your `dataviz-exercises` project and use a different data set, for example `ggplot2::mpg` or `ggplot2::diamonds`. Class time is for working on exercises; please **do not** use class time to set up your project.

Data

Identify a data file from `data/` to work with. Consider discussing with your collaborators which data to use.

Git Branch

Ensure you are on your personal branch and not working on another branch. *Hint:* `git branch`

Part 1: Creating a Point Plot: Mapping a Numeric Variable to an Aesthetic

Using a data set, identify some relevant numeric and non-numeric variables. Create a point plot with a numeric variable along the y-axis and a factor/character variable along the x-axis. The character variable could represent a grouping variable from a previous exercise. Map a numeric variable to an aesthetic other than x or y. Do not map to color for this step.

1. Create an .R script file (Recommendation: Start with `starter_script_file.R`)
2. Name the file appropriately based on the goal of the three parts of the exercise (review briefly). Add either your name or initials as a prefix to the file name. Use lower case characters and do not use spaces.
3. In a “library” section of your script, load your libraries.
4. In a “data-reading” section of your script, write code to read your data. Assign the data frame an object name that unambiguously represents the data you are reading. If you are not using `{here}`, make sure you do. If you are using an aggregated data file, good practice is to `source()` your aggregation script that produces your aggregated data file so that the plot you make will be based on most-recent aggregation.
5. Take inventory of your data data frame structure so that you can identity what variables are numeric and non-numeric. Just because variables “look” a certain way to the eye, they may not be the data type you believe they are. Verify. *Hint: glimpse()*.
6. In a “plotting” section of your script, start with your data frame and code a piped workflow to create a point plot. Map numeric variables to the `x` and `y` aesthetics inside of `ggplot()` before passing to your `geom_*()`. Check your plot to ensure it looks like you intend.
7. Modify your code to map a *numeric* variable to an aesthetic other than `x` or `y`.
8. Investigate the plot visually (not quantitatively) to determine whether there are associations or patterns in the data worth noting to investigating more thoroughly. If so, make some notes to reference later. A good recommendation is to create a simple `<my-name>_project_notes.txt` file and bullet point your notes there. If you are wondering where to save the file, you can make a directory like `notes/` at your project root directory and save it there so it has a home. Whatever you decided, your team should agree so that your project does not have multiple directories with similar information. The project should be kept tidy.
9. Re-save your plot script.

Part 2: Creating a Point Plot: Mapping a Non-Numeric Variable to Color

Using either the same variables or a different set of variables, create a point plot. Map a non-numeric variable to the *color* aesthetic.

1. In a new “plotting” section of your `.R` script, describe what this new code will do. But see Pro Tip below.
2. Write code to pipe your data frame and create a *new* point plot with a non-numeric variable mapped to the color aesthetic.
3. Re-save your plot script file.
4. Investigate the plot visually (not quantitatively) to determine whether there are associations or patterns in the data worth noting to investigating more thoroughly. If so, make some notes to reference later.
5. Consider whether the plot appears too busy and complicated. Later exercises will work on fixing those issues. Make a list of suggestions to yourself to reference later.

Pro Tip: For your team project, I advise strongly that you do not create multiple plots in a single script as code failures for some plot may cause code failures for a subsequent plot. A safer approach is to create separate scripts for each plot so that you can compartmentalize problems that might arise with file conflicts.

Hint: If you have not done so, now may be the time for the team or a specific individual to start determining the colors you would use for your project. Perhaps there are official industry colors or something else. Decide whether such a palette would be useful for your visualizations; not all colors and color combinations are useful. Customizing colors will be addressed in later exercises.

Part 3: Creating a Point Plot: Mapping a Different Non-Numeric Variable to Color

Using the same variables used in Part 2, create a point plot that maps a *different* character/factor variable. Ideally, choose a variable that contains a different number of unique groups. Your *color* aesthetic will allow you to explore differences in the data as well as see the differences in the color palette applied to that specific variable. If you do not have a second non-numeric variable, either `mutate()` one based on an existing variable or a `filter()` your data frame to keep or remove a group so that you have a different number of groups before you pass it to `ggplot()`.

1. In a new “plotting” section of your .R script, describe what this new code will do.
2. Write code to pipe your data frame and create a *new* point plot with a different or modified non-numeric variable mapped to the color aesthetic.
3. Investigate the plot visually (not quantitatively) to determine whether there are associations or patterns in the data worth noting to investigate more thoroughly. If so, make some notes to reference later.
4. Consider whether the plot appears too busy and complicated. Would changing something to the plot make it easier to understand? Consider other ways to visualize the data. Make a list of suggestions to reference later.
5. Re-save your plot script file.

Part 4: Thinking Through Git Steps

These files are not ready for your project, so do not push them to GitHub. However, think through the steps that you would take to do so.